

FidelityMCTS: Selective Predictive and Executable Evidence for Stateful Agent Planning

Anonymous Authors

Draft research proposal and system specification

September 2026

Abstract

Stateful agents can obtain provisional predictions from learned world models and stronger evidence from isolated executable environments. The useful research question is no longer whether to allocate test-time compute in general, but whether a controller can acquire those two kinds of evidence selectively while preserving repeated-run reliability under measured resource and risk limits. We refine FIDELITYMCTS around that question. The target controller selects a branch, evidence source, and stop decision; model, token, horizon, and verification choices are later extensions that must be isolated empirically. The implementation supplies a modular Gymnasium-compatible MCTS kernel, resource accounting, multi-fidelity frontier valuation, verified replay, simple discrepancy calibration, and binary learned routing. It does not yet implement multi-model tree transitions, full joint routing, a hard episode-level risk envelope, or an evaluated self-improvement loop. We therefore make MCTS a secondary controller hypothesis: it must outperform a capacity- and budget-matched non-tree policy after a sequential benchmark shows oracle headroom. This research-status draft specifies that narrower hypothesis, modern baselines, validity gates, and versioned evaluation requirements. It reports no confirmatory benchmark result.

Research status (2026-09-10). The classical kernel and evidence-routing infrastructure exist. The FrozenLake and one-decision SQLite fixtures are engineering diagnostics, not evidence for the hypothesis. No sequential agent adapter, valid confirmatory protocol, or confirmatory result exists.

1 Introduction

Test-time computation has become an important axis of capability. A system may sample several solutions, revise a trajectory, search a tree, call a stronger model, or execute a candidate in a sandbox. Recent agent work studies parallel sampling, sequential revision, dynamic planning, per-step allocation, and agentic orchestration [13, 16, 20, 22, 36]. A broad “learning when to think” claim is therefore both timely and crowded.

MCTS remains a plausible substrate for adaptive allocation. Its central purpose is to spend simulations

non-uniformly across actions [4, 14]. Yet the simulation primitive is normally fixed: each rollout is treated as if it came from the same environment model at the same price. Neural MCTS changes the source of priors or values [25, 27, 28], but it does not usually make simulator identity, executable evidence, and stopping a joint branch-level decision. Explicit static and dynamic values of computation have also already been studied within MCTS [26]. Value of computation or MCTS alone is not the novelty.

Model routing balances quality and cost [19]; newer systems jointly control model and output length [34], choose metacognitive operations [18], or let an orchestrator gather and synthesize evidence [22]. The remaining distinction is stateful and environmental: “trust a prediction for routine branches, execute two disputed branches against disposable state, then stop.”

The required resources are becoming practical. Language world models can simulate agentic transitions [12, 37], while code- and database-backed environments provide more expensive executable transitions and programmatic verifiers [32]. Stateful tool benchmarks such as τ^2 -bench expose dual-control interactions and verifiable world state [3]. BrowserGym and WorkArena expose web and knowledge-work tasks through gym-like interfaces [7, 15]. The research opportunity is to test selective acquisition of executable evidence while holding the agent harness and context policy fixed. Context management can itself drive long-horizon gains [17], so it cannot remain an unrecorded implementation detail.

The target FIDELITYMCTS separates an object-level environment action from a meta-level evidence query. A query initially names a branch and either a predictive or isolated executable source; a stop action commits to the best current task action. The experiment constrains episode-level money, tokens, wall time, executable calls, and action risk. Finer-grained model, token, horizon, and verification choices are admitted only through separate, identifiable extensions.

The intended contributions are:

1. a falsifiable hypothesis about branch-local acquisition of predictive versus independently verified executable evidence in stateful tasks;
2. a controller comparison that treats MCTS as a candidate and requires capacity-matched non-tree, query-

level, fixed, revision, and sampling baselines;

3. an implemented Gymnasium-compatible kernel and evidence ledger whose present boundary is stated separately from target architecture; and
4. an experiment contract that freezes harness, context, action, environment, grader, verifier, model, and resource-accounting versions.

The contribution is *not* the number of supported algorithms, value of computation in MCTS, or generic “when to think” control. Classical variants are reusable infrastructure. The proposed scientific claim concerns selective predictive/executable evidence in a sequential, externally verified setting.

2 Background and Related Work

2.1 Classical and Bayesian MCTS

UCT applies upper confidence bounds to tree selection [14]. MCTS has since accumulated selection, expansion, rollout, backup, parallelization, transposition, and domain-knowledge variants [4]. Crazy Stone introduced soft selectivity and studied mean, robust, and mixed backups [5]. Robust max uses the value of the most-visited move, avoiding a noisy maximum over empirical values.

Posterior-sampling approaches represent uncertainty directly. Bayesian reinforcement learning with MCTS can maintain Dirichlet transition beliefs [31]; posterior-sampling MCTS can use conjugate reward and transition models or root-sampled dynamics [2]. FIDELITYMCTS uses these mechanisms as possible uncertainty representations, but its compute policy also chooses *which evidence source* to sample.

2.2 Neural search and language-model planning

AlphaGo combines policy priors, value estimates, rollouts, and PUCT [27]; AlphaGo Zero removes rollouts and directly backs up a policy-value network [28]. MuZero plans with a learned latent dynamics model [25]. More recently, AlphaZero-like search has guided LLM decoding and supplied training targets [9]; MCTS has also been used to collect preference data for online optimization [33]. Tree-structured generative planning continues to improve with inference compute [35].

These results support search as a policy-improvement operator, but do not imply that a tree is the best controller for agentic test-time compute. Recent coding results obtain gains from trajectory summarization, tournament selection, and sequential refinement without the proposed environment tree [13]. A non-tree controller is therefore a required scientific comparison rather than a weaker implementation baseline.

2.3 Routing, metareasoning, and world models

RouteLLM learns cost-quality routing from preferences [19]. R2-Router jointly selects a model and output-length budget [34]. Rational metareasoning studies the value of computation (VOC), a long-standing limited-rationality concept [24], for selective LLM reasoning [6]. Meta-BAMDPs extend metareasoning to uncertain environments [11]; recent metacognitive controllers choose operations such as expansion, pruning, repair, and stopping [18].

Dynamic planning agents choose whether to plan before individual actions [20], and CATTS adapts per-step sampling from vote uncertainty in web agents [16]. ATLAS allows a stateful orchestrator to gather further solver evidence, choose solver and reasoning effort, stop, and synthesize [22]. Together with explicit VOC policies for MCTS [26], these systems invalidate novelty claims based only on adaptive compute, stopping, or branch granularity.

The narrower FIDELITYMCTS hypothesis adds a specific evidence intervention: selecting between a learned environment prediction and an independently executed disposable state transition within a sequential task. Whether MCTS is useful for that intervention remains empirical.

Language world models now predict state transitions in agent environments [37]; WebDreamer uses an LLM as both world model and value function for web planning [12]. Agent World Model constructs 1,000 code- and database-backed executable environments [32]. This creates a useful asymmetry: learned models provide cheap speculative breadth, while executable environments provide costlier, higher-confidence evidence. C-MCTS also motivates low- and high-fidelity simulators for safe planning, though it uses the high-fidelity model offline to train a safety critic [21]. FIDELITYMCTS instead proposes online, selective executable-evidence allocation. Qwen-AgentWorld reports that uncontrolled simulation can harm some agent tasks, which reinforces the need to measure decision impact rather than raw prediction error alone [37].

2.4 Preference learning and causality

Search can optimize a reward or preference model and can generate targets for methods such as direct preference optimization [23, 33]. It does not eliminate the need to elicit human objectives. Indeed, stronger search can exploit reward-model errors more aggressively.

Selective routing creates observational data: expensive outcomes are observed where the current router chose to query. Causal routing addresses decision learning from such logged outcomes [30], while doubly robust estimators combine propensity and outcome models [8]. Causal world models may also improve long-horizon language-agent planning [10]. We adopt a narrow position: causal

evaluation is valuable for routing logs and structured tool effects, but a universal causal world model is not a v1 prerequisite.

3 Problem Formulation

This section defines the target research intervention, not the complete current API. The implemented router is binary (cheap versus accurate) with fixed token and depth settings; verification is coupled to accurate evaluation. Its learned target is absolute cheap-verified discrepancy rather than the decision utility in Equation 4.

3.1 Object-level environment

Let an environment be a finite-horizon MDP or information-state process with states $s \in \mathcal{S}$, task actions $a \in \mathcal{A}(s)$, rewards r , and transition kernel P . In a partially observed setting, s denotes the planner’s information state, not inaccessible hidden state. A task policy π_A ultimately selects a real action for the live environment. In realistic tool environments, $\mathcal{A}(s)$ may be too large or structured to enumerate. A proposal policy must then supply candidate actions together with proposal provenance and cost; macro-actions or progressive widening become part of the frozen controller condition.

We assume a portfolio $\mathcal{M} = \{m_1, \dots, m_K\}$ of generative models. A member may be a learned transition/value model, LLM, reversible Gymnasium environment, executable sandbox, or verifier. Querying m at state-action pair (s, a) produces evidence

$$o \sim p_m(o \mid s, a, b, d),$$

where b is a token/sample budget and d is a rollout horizon. Evidence may contain a next state, reward, value estimate, uncertainty, terminal flags, and provenance. Each query incurs a resource vector

$$g(o, m) = (c_{\text{money}}, c_{\text{token}}, c_{\text{time}}, c_{\text{accurate}}, c_{\text{risk}}).$$

No portfolio member is declared ground truth by type. “Accurate” is a relative fidelity label established empirically. Real outcomes and deterministic verifiers have distinct provenance.

3.2 Compute actions

At meta-history $h \in \mathcal{H}$, containing the partial search graph and all evidence, the compute policy π_C chooses

$$c = (s, a, m, b, d, v) \in \mathcal{C}(h),$$

where v indicates requested verification. It may instead choose STOP, after which $\pi_A(h)$ is executed.

The first confirmatory intervention restricts this space to branch, evidence source, and stop. Token, horizon, verifier, context, and multi-agent-topology control are later

factors; jointly varying them in the first experiment would prevent attribution.

The joint objective can be expressed as a constrained problem,

$$\max_{\pi_A, \pi_C} \mathbb{E}[G(\pi_A; h_{\text{stop}})] \quad (1)$$

$$\text{s.t. } \mathbb{E}\left[\sum_t g_j(c_t)\right] \leq B_j \quad \forall j, \quad (2)$$

$$\Pr(\text{unsafe action}) \leq \epsilon, \quad (3)$$

or as a Lagrangian utility

$$J = \mathbb{E}\left[G - \lambda^\top \sum_t g(c_t) - \kappa \text{Risk}\right].$$

We report Pareto frontiers because a single set of multipliers can conceal important quality-cost-risk trade-offs. All constraints in the primary experiment apply to the complete task episode. A fresh allowance per planner call is a diagnostic configuration, not an episode-level budget guarantee.

3.3 Discrepancy and value of computation

For a cheap model m and executable reference H , define paired discrepancy

$$\delta_m(s, a, h) = y_H(s, a) - \hat{y}_m(s, a, h).$$

A discrepancy model predicts a distribution over δ_m , rather than forcing cheap and executable estimates into one undifferentiated mean.

The ideal expected value of computation (EVC) is

$$\text{EVC}(c \mid h) = \mathbb{E}_{o_c}[U(\pi_A(h \cup o_c)) - U(\pi_A(h))] - \lambda^\top \mathbb{E}[g(c)]. \quad (4)$$

Exact EVC is generally intractable because it integrates future search changes. We study analytic approximations, contextual bandits, and learned EVC models. Before fitting such a model, the study must define a decision-level target and show that an oracle selective policy has headroom over simpler controls.

4 FidelityMCTS

The following is target architecture. In the current implementation, outer MCTS transitions are produced by one simulation model and the adaptive portfolio evaluates already-created frontier actions. Portfolio `next_state` observations are not incorporated into the search graph. Consequently the delivered component is multi-fidelity frontier valuation, not yet multi-model transition search.

4.1 Search graph

We represent search as a state-action outcome graph. A state node stores state identity and expansion metadata. An action edge stores $N(s, a)$, $W(s, a)$, $Q(s, a)$, prior $P(s, a)$, optional Bayesian or AMAF statistics, and model-indexed evidence. Outcome links represent stochastic reward/next-state realizations. An iteration-local search path replaces a canonical parent pointer, allowing transpositions without ambiguous backup.

This representation is important for multi-fidelity evidence. Querying an accurate simulator does not overwrite a cheap prediction. Both remain attached to the edge with model version, uncertainty, and measured cost.

4.2 Two coupled policies

The tree policy selects a promising frontier under ordinary MCTS statistics. At that frontier, the compute policy repeatedly chooses evidence queries until a stop policy fires. The evaluator aggregates current evidence, and a backup operator propagates the resulting return. Thus UCT, PUCT, Thompson sampling, direct neural evaluation, rollouts, and alternative backups remain injected components rather than separate engine implementations.

Figure 1 shows the system. The core design constraint is that the live environment is never mutated by speculative search.

4.3 Routing and stopping

A reference score for a feasible compute action is

$$S_\theta(c, h) = \widehat{\Delta U}_\theta(c, h) - \lambda^\top \widehat{g}_\theta(c, h) - \kappa \widehat{\rho}_\theta(c, h), \quad (5)$$

where $\widehat{\Delta U}$ predicts decision improvement, $\widehat{g}$ resources, and $\widehat{\rho}$ risk. In the target controller, unaffordable route-action pairs are masked before scoring, and a hard risk ceiling is enforced independently of the score. The router chooses $\arg \max_c S_\theta(c, h)$ only when the maximum is positive; otherwise it stops. The current binary router instead selects a fidelity and then attempts to reserve its quote; an unaffordable choice stops rather than trying the best affordable alternative, and the current ledger has no hard risk ceiling.

Router features include search depth, value gaps, visits, predictive variance, model disagreement, expected cheap-to-accurate discrepancy, novelty, branch impact, reversibility, verifier availability, remaining budgets, and service load. The action may be factorized into branch, model, token budget, horizon, sample count, and verifier. This permits new portfolio members without retraining a flat classifier from scratch.

These factorized choices are proposed, not delivered. Current learned-route features include remaining-budget fields, but training only at the largest SQLite budget makes those fields constant; search depth is also fixed at zero in that fixture.

We include simple uncertainty-threshold and fixed-cascade policies as strong engineering baselines. A learned router earns its complexity only if it outperforms them under matched budgets.

4.4 Evidence aggregation and backup

An evidence aggregator produces a return distribution, not only a scalar. Candidate implementations are:

- verified-first selection;
- Bayesian fusion with model-specific likelihoods;
- cheap prediction plus discrepancy posterior;
- ensemble disagreement with conservative lower bounds;
- direct neural value or mixed value/rollout estimates.

Only a simple verified-first/discrepancy path is currently exercised. Its verified source is the first configured accurate-fidelity model, so heterogeneous accurate sources require an explicit source-selection rule before evaluation.

The backup interface then chooses mean, distributional, robust, or mixed propagation. For Crazy Stone compatibility,

$$V_{\text{robust}}(s) = Q\left(s, \arg \max_a N(s, a)\right),$$

and

$$V_{\text{mix}}(s) = (1 - \alpha_s)V_{\text{mean}}(s) + \alpha_s V_{\text{robust}}(s),$$

where α_s can depend on visits and variance. Alternating-agent value perspective is configured explicitly.

4.5 Budgeted search procedure

The ledger reserves a conservative model quote before a query and commits measured usage afterward. A model that exceeds its quote triggers a fault policy rather than silently breaking the search budget.

Algorithm 2 is not the current end-to-end execution path. The current adaptive component values an MCTS-created frontier but does not use portfolio observations as tree transitions. Each planning call also receives a new ledger; the FrozenLake diagnostic sums those calls after the fact rather than enforcing one episode envelope.

4.6 Verified replay and proposed self-improvement

Whenever executable or real evidence becomes available, the system stores

$$(h, s, a, m, \hat{y}_m, \hat{\sigma}_m, c, p(c \mid h), y_H, g, \text{versions}).$$

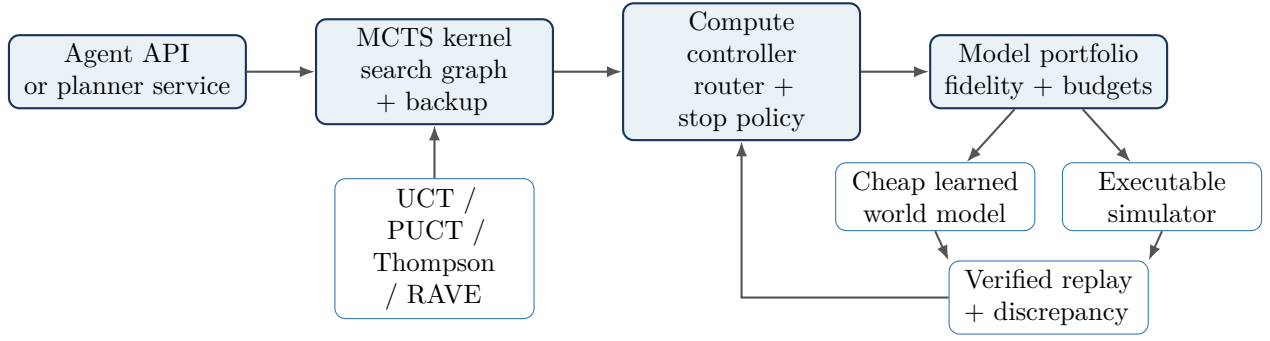


Figure 1: FIDELITYMCTS separates MCTS mechanics, algorithmic rules, and meta-level compute allocation. Cheap and executable evidence are paired in verified replay.

Algorithm 1: Adaptive multi-fidelity MCTS

1. Synchronize or create the root search graph.
2. While the resource ledger permits an iteration:
 - (a) select a path using the injected tree policy;
 - (b) expand a legal frontier;
 - (c) build a router context from evidence and budgets;
 - (d) query feasible compute actions until the router stops;
 - (e) aggregate evidence into an evaluation distribution;
 - (f) back up along the explicit iteration path;
 - (g) append a structured trace and measured usage.
3. Select a root task action under the configured decision rule.

Figure 2: Algorithm-name conditionals are unnecessary. AlphaGo Zero uses direct neural evaluation plus a no-rollout policy; UCT uses rollouts; adaptive variants insert model queries before evidence aggregation.

The target record supports four distinct objectives:

$$\mathcal{L}_{\text{value}} = \ell(\hat{y}_m, y_H), \quad (6)$$

$$\mathcal{L}_{\text{disc}} = \ell(\hat{\delta}_m, y_H - \hat{y}_m), \quad (7)$$

$$\mathcal{L}_{\text{route}} = -\widehat{\text{EVC}}(c, h), \quad (8)$$

$$\mathcal{L}_{\text{cal}} = \ell_{\text{proper}}(\hat{p}_m, y_H). \quad (9)$$

Updates are staged: collect with frozen versions, validate, train candidates, evaluate offline, run randomized canaries, and promote only after quality, calibration, cost, and safety gates pass.

Persistent replay, online discrepancy updates, audit logging, and estimator utilities exist. No evaluated multi-round train-deploy-collect loop yet shows improvement, stability, or resistance to selection bias. The SQLite OPE utility uses selected verification events with an effectively deterministic target; its “online randomized” comparison is the same conditional pass rate, so it cannot validate router ranking.

Search can also produce visit distributions or verified pairwise rankings for distillation and preference optimiza-

tion [9, 33]. This can replace the policy-optimization step of some RLHF pipelines, but not human preference elicitation or reward validation. Model-only outcomes remain labeled synthetic.

4.7 Causal logging and evaluation

Because the router decides what to verify, verified replay is selected data. We log the probability of every chosen route and allocate bounded randomized audit traffic. For a deterministic target router π , a doubly robust estimate has the form

$$\hat{V}_{\text{DR}}(\pi) = \frac{1}{n} \sum_{i=1}^n [\hat{r}(x_i, \pi(x_i)) + \frac{\mathbf{1}\{c_i = \pi(x_i)\}}{p_i} (r_i - \hat{r}(x_i, c_i))] \quad (10)$$

Overlap, stable treatment definitions, and sensitivity analyses are required. Causal claims are limited to interventions the system can define and support.

5 Software Architecture

5.1 MonteCarloGym kernel

MONTECARLOGYM provides a stable agent interface:

```

action = agent.compute_action(
    sim_env, observation)

```

The user then calls the normal Gymnasium `env.step(action)` and passes the real transition to `agent.observe`. Gymnasium standardizes environment interaction [29], but generic generative-model protocols keep the kernel usable outside Gym.

The environment wrapper currently supports native snapshot/restore or safe deep copy. Clone-state, deterministic event reconstruction, and external-model adapters are target strategies, not delivered implementations. A transaction restores internal state, wrapper state, RNG streams, episode counters, and terminated/truncated flags even after exceptions. Environments that cannot satisfy this contract fail fast.

The search graph stores statistics on action edges and represents stochastic outcomes explicitly. Transposition tables use adapter-provided state keys. Tree reuse re-roots only after a matching real transition and invalidates incompatible model or state versions.

5.2 Dependency injection and presets

The engine depends on protocols for tree policy, expander, evaluator, rollout policy, backup, model portfolio, compute router, stop policy, reward source, state codec, and trace sink. Table 1 summarizes classical presets. They share the same engine.

5.3 Deployment and open-source surfaces

The dependency-light Python core is packaged for a future PyPI release; this draft does not assert that it is published. A containerized planner service and HTTP/gRPC/MCP interfaces are also planned rather than implemented. Thin TypeScript adapters should expose an explicit `plan_and_execute` operation rather than assume a late tool hook can control earlier model selection. Router and discrepancy checkpoints, model cards, and permitted datasets can be released through model/data hubs.

Any future production deployment should separate search coordinators, batched model gateways, simulator workers, replay storage, tracing, and asynchronous training. Every request has a budget envelope and idempotency key. Speculative execution uses a sandbox or dry run; irreversible or externally visible actions require an approval boundary.

6 Experimental Protocol

No confirmatory hypothesis or seed set is currently registered. A fresh study first tests whether a selective predictive/executable policy has oracle headroom on a genuinely sequential task. Only then does it compare end-to-end controllers under the same task distribution, action proposer, context policy, harness, portfolio, external-action policy, and hard episode budget. The task instance is the independent unit; model samples, candidate orderings, and random seeds within one task are repeated measurements.

6.1 Benchmark ladder

L0 and L1 validate identifiability and kernel behavior. The existing SQLite fixture is L2a: one decision, four task templates, normalized costs, and no outer-MCTS execution. It cannot test sequential planning, tree reuse, or episode budgets. L2b is the first eligible primary study, with a pinned τ^2 -bench-style stateful tool environment or

Agent World Model environment [3, 32], an action proposer, and independent state verification. L3 is a later replication using a pinned BrowserGym/WorkArena-like or desktop environment [7, 15].

6.2 Baselines

Required comparisons are: direct/no-search; cheap-only; accurate-only; a fixed budget or top- k cascade; random escalation matched to executable-call rate; an uncertainty threshold; a query-level router; joint model/length routing; a capacity-matched direct non-tree adaptive controller; parallel best-of- N with listwise verification; sequential revise/reflect; native reasoning-effort control; verified-state management; and the MCTS controller. Agentic test-time-compute work shows that sampling, revision, diversity, aggregation, and trajectory reuse are consequential controls [13, 36]. Multi-agent decomposition is included only where the task supports parallelism and only with matched cost.

An oracle is a design diagnostic, not automatically an upper bound. If portfolio order or affordability constrains it, it is labeled a “budget-limited counterfactual.” MCTS is evaluated only when a truly budget-informed oracle establishes headroom.

6.3 Metrics and primary analysis

The primary quality measure is terminal task state from an independent, versioned verifier, not the agent’s completion claim. False-success work shows that confident completion can disagree sharply with environment state [1]. We report $\text{pass}@k$, pass^k , verifier false-positive/negative rates, stale-state errors, assertion coverage, side-effects, and infrastructure failures separately from agent failures.

Resource measures are real money, input/output/context tokens, CPU/GPU time, executable calls, concurrency-adjusted wall time, memory, approval requests, and action risk. We report medians and tails/CVaR in addition to means. Diagnostics include value error, interval coverage, calibration, discrepancy, escalation precision/recall, stopping regret, and offline-evaluation bias.

The primary result is a repeated-run verified-success-cost-risk frontier over prespecified episode budgets. We report paired hypervolume differences under a frozen reference point, success at matched cost, cost at matched success, and area under the budget-performance curve. A single weighted aggregate is secondary.

6.4 Hypotheses and ablations

The candidate primary hypothesis is that branch-local selection between predictive and isolated executable evidence improves the verified-success-cost frontier over the strongest fixed, query-level, and direct non-tree controls.

Table 1: Implemented and tested classical compatibility presets in the current kernel.

Preset	Selection	Evaluation	Statistics	Backup/sharing
UCT	UCB1	random or heuristic rollout	edge N, W, Q	mean
DNG-MCTS / MCBRL	Thompson or root sampling	sampld re-ward/transition model	Normal-Gamma and Dirichlet	Bayesian update
Crazy Stone	soft/Boltzmann	rollout	visits, mean, variance	mean, robust, or mix
AlphaGo APV	PUCT	value network plus fast rollout	N, W, Q, P	mean, neural batching
AlphaGo Zero	PUCT	direct policy-value network	N, W, Q, P	mean, no rollout
RAVE / MAST	blended UCT/RAVE	globally biased rollout	direct, AMAF, global move values	direct+AMAF / base backup

Table 2: Planned benchmark ladder.

Level	Domain	Cheap	Accurate
L0	synthetic	biased oracle	truth oracle
L1	Gym env	learned dynamics	cloned env
L2a	local fixture	learned score	SQLite clone
L2b	tool agent	world model	code/DB clone
L3	web/GUI	DOM/world	sandbox

The conditional MCTS hypothesis is that the tree controller adds improvement over a capacity- and budget-matched non-tree controller. Secondary candidates test decision-aware discrepancy versus raw prediction error, and non-inferiority of learned stopping at lower resource use. Replay improvement and off-policy router ranking remain future questions until a valid multi-round loop and overlapping randomized behavior policy exist.

Ablations must alter one operative mechanism. The SQLite labels “no verification” and “cheap only” currently coincide; “no discrepancy” still updates the discrepancy model; and “no tree reuse” is inapplicable. Those labels cannot support causal component claims and must not be copied into the fresh protocol.

6.5 Statistics and artifacts

Task instances are paired across methods. Resampling and inference preserve task/family clusters and never flatten seeds or budget rows into independent observations. Sample sizes follow a tracked pilot with raw outputs and an auditable effect-size lineage. The confirmatory plan prespecifies paired hypervolume inference, task-stratified bootstrap intervals, multiplicity control, and an explicit non-inferiority margin. The existing SQLite candidate’s “32 repetitions” are four tasks under eight candidate-order permutations and do not provide 32 independent units; its embedded power calculation is invalidated.

Each run releases its resolved configuration, source revision, artifact hashes, environment, harness, context, action, grader, verifier, and model versions, reasoning settings, seeds, structured decision and episode ledgers, per-

episode JSONL, failures, and analysis. Reanalysis verifies every input and manifest hash. Provider and infrastructure non-determinism are measured with repeated trials across time/load strata. All current local fixtures remain software or mechanism diagnostics, not reported confirmatory results.

7 Discussion

7.1 Why this may work

The hypothesis exploits two forms of heterogeneity. First, learned-model error is structured: a predictive source may be adequate for routine transitions but fail on novel, ambiguous, or safety-critical states. Second, evidence has decision-dependent value: an additional observation matters only when it can change an action or reduce unacceptable risk. EVC gives a language for the second effect, but raw prediction discrepancy is not itself EVC.

A tree may add a third form—unequal branch importance—but only if there is enough branching, horizon, and affordable executable headroom. If a direct controller captures the same evidence boundary, MCTS overhead is unjustified.

7.2 Relationship to RLHF

Predictive search can serve as policy improvement at inference time or produce targets for distillation/preference optimization. In this limited sense it can replace “the RL step” of an alignment pipeline. It cannot determine what humans want, validate a reward model, or guarantee safety. Search may amplify reward hacking by finding adversarial high-score trajectories. We therefore retain reward provenance, component scores, uncertainty penalties, independent verifiers, and human approval for consequential actions.

7.3 Why off-policy evaluation is deferred

Off-policy correction addresses a concrete problem: the next router is trained and selected from outcomes ac-

quired by the previous router. Randomized audits and propensity-based estimators can reduce selection bias, but only with stable treatments, overlap, and an independent online target. The current SQLite analysis does not meet that bar. Structured causal world models may help where variables are stable and intervenable; universal causal modeling is outside the first experiment.

8 Limitations and Risks

EVC estimation. The value of a computation depends on counterfactual future search, making supervision sparse and policy-dependent. Simple threshold routers may remain competitive. Oracle-router experiments are needed to quantify available headroom.

Controller necessity. MCTS can be the wrong abstraction when actions are open-ended, proposal quality dominates, or horizon is short. Capacity-matched non-tree controllers, action proposal costs, and an oracle-headroom gate are necessary to distinguish a search benefit from routing alone.

High fidelity is not ground truth. Executable synthetic environments can omit production behavior. Browser sandboxes can differ from live sites. We must report simulator-real discrepancy and use real outcomes only where safe and consented.

Verifier and benchmark drift. Programmatic graders can be incomplete, LLM judges can follow surface cues, and benchmark releases can alter task semantics. Verifier identity, assertion coverage, benchmark and grader versions, false-positive/negative audits, and environment-state evidence are part of every result.

Harness and context confounding. Agent harnesses and context management can materially change long-horizon outcomes [17]. The primary comparison holds them fixed and logs them. Jointly optimizing context or agent topology is later work.

Distribution shift and feedback loops. Verified replay is concentrated near the router’s boundary. Without exploration and correction, the system may become confidently blind elsewhere. Jointly updating world model and router can destabilize targets.

Safety and security. Tree search can generate many tool actions, increasing prompt-injection and credential exposure. Speculation must be isolated; logs must redact secrets; side effects must be approval-gated. A sandbox reduces but does not eliminate risk.

Overhead. Maintaining uncertainty, discrepancy, traces, and routing can cost more than it saves on easy tasks. No-search and fixed-cascade baselines are therefore essential.

Research breadth. The target architecture spans MCTS, routing, world models, causal evaluation, and agent systems. The first eligible study fixes one predictive/executable pair and one evidence action. Joint model/token/depth/verifier routing, multi-agent topology, and self-improvement are explicitly deferred.

Current evidence. The implementation has no eligible sequential study. SQLite Phase 5A is a one-decision instrumentation diagnostic with four task templates, non-independent order permutations, non-isolated ablations, and a budget-limited “oracle.” It cannot support the proposed hypothesis.

9 Open-Source Plan and Broader Impact

The current codebase separates a minimal MONTECARLOGYM kernel from optional Gymnasium support. PyTorch, Transformers, browser, service, registry, and archival-release surfaces are plans, not delivered products. Any public research artifact must retain its exact protocol, inputs, hashes, and validity status.

Positive uses include lower-cost planning, reproducible MCTS research, safer sandbox-first agents, and better resource accounting. Negative uses include more capable automation of harmful tasks and reward-model exploitation. Adapters should default to read-only/dry-run operation, use scoped credentials, and expose both safety and cost telemetry. Training data and model-generated traces must be labeled by provenance.

10 Conclusion

The fit-for-purpose review narrows FIDELITYMCTS from generic joint test-time-compute control to selective acquisition of predictive and executable evidence in stateful agent tasks. The premise remains plausible and increasingly practical, but broad novelty has been overtaken by dynamic planning, routing, and agentic orchestration work. The classical kernel and evidence infrastructure remain useful; the old empirical claim does not.

The next paper succeeds only if a fresh, versioned sequential study first shows oracle headroom, then improves repeated-run independently verified success against modern matched-budget controls. MCTS receives a separate conditional test against a capacity-matched non-tree controller. Failure at either gate is a result and a reason to simplify or redirect the project.

A API Contracts

Generative model. A model exposes stable identity, fidelity, a conservative pre-call resource quote, and an evaluation method. Its observation includes value, variance, measured cost, tokens, latency, risk, optional next state, terminal flags, and metadata.

Planner. The target planner receives a state, model portfolio, hard episode envelope, remaining resources, and seed. It returns a task action, predicted value, measured decision and cumulative episode usage, and an auditable structured trace. Current APIs enforce planning-call rather than episode budgets.

Reward source. Rules, environment rewards, preference models, humans, and verifiers implement one scoring protocol. A score carries scalar value, named components, verification status, source, version, assertion coverage, and retained evidence. The richer verification fields are a target schema extension.

Replay. A verified record pairs cheap predictions with executable or real outcomes and includes router propensity and full environment, prompt, model, verifier, and resource versions. Synthetic, executable, and real provenance are never collapsed. The current replay schema is narrower and must be migrated before a confirmatory run.

B Implementation Invariants

1. Snapshot/restore reproduces environment and RNG state.
2. Planning cannot mutate the live environment.
3. Edge visits equal completed backups.
4. Configured planning-call resource limits are never exceeded, including failures.
5. Termination and truncation remain distinct.
6. Robust max selects the most-visited child.
7. Imaginary Bayesian transitions do not update the real root belief.
8. RAVE direct and AMAF updates are not double counted.
9. Accurate evidence augments rather than silently overwrites provenance.
10. Every learned route used for causal evaluation has a logged propensity.

Hard episode budgets, a risk ceiling, feasible-action masking, and complete versioned replay are target invariants, not current guarantees.

C Claim Readiness

Before replacing proposal language with empirical claims, require: demonstrated oracle headroom; a sequential executable task; independent terminal-state verification; a frozen harness, context policy, action proposer, environment, grader, verifier, model, and reasoning configuration; hard episode budgets; modern matched-budget and capacity-matched non-tree baselines; isolated ablations; independent task units and clustered analysis; repeated reliability, tail-risk, verifier, and infrastructure reporting; full artifact hashes; and a fresh protocol frozen before confirmatory data collection.

References

- [1] Laksh Advani. From confident closing to silent failure: Characterizing false success in LLM agents. *arXiv preprint arXiv:2606.09863*, 2026. Accepted to FAGEN at ICML 2026.
- [2] Aijun Bai, Feng Wu, and Xiaoping Chen. Posterior sampling for monte carlo planning under uncertainty. *Applied Intelligence*, 48:4998–5018, 2018. doi: 10.1007/s10489-018-1248-5.
- [3] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -Bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- [4] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012. doi: 10.1109/TCIAIG.2012.2186810.
- [5] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *Computers and Games*, pages 72–83. Springer, 2006.
- [6] C. Nicolò De Sabbata, Theodore R. Sumers, Badr AlKhamissi, Antoine Bosselut, and Thomas L. Griffiths. Rational metareasoning for large language models. *arXiv preprint arXiv:2410.05563*, 2024.
- [7] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. WorkArena: How capable are web agents at solving common knowledge work tasks? In *International Conference on Machine Learning*, pages 11642–11662. PMLR, 2024.
- [8] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *International Conference on Machine Learning*, pages 1097–1104, 2011.

- [9] Xidong Feng, Ziyu Wan, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. Alphazero-like tree-search can guide large language model decoding and training. *International Conference on Machine Learning*, 2024.
- [10] John Gkountouras, Matthias Lindemann, Phillip Lippe, Efstratios Gavves, and Ivan Titov. Language agents meet causality: Bridging LLMs and causal world models. *International Conference on Learning Representations*, 2025.
- [11] Prakhar Godara and Tilman Diego Alemán. Metareasoning in uncertain environments: A meta-BAMDP framework. *arXiv preprint arXiv:2408.01253*, 2024.
- [12] Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, et al. Is your LLM secretly a world model of the internet? model-based planning for web agents. *arXiv preprint arXiv:2411.06559*, 2025.
- [13] Joongwon Kim, Wannan Yang, Kelvin Niu, Hongming Zhang, Yun Zhu, Eryk Helenowski, Ruan Silva, Zhengxing Chen, et al. Scaling test-time compute for agentic coding. *arXiv preprint arXiv:2604.16529*, 2026.
- [14] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.
- [15] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayaneg, Lawrence Keunho Jang, et al. The BrowserGym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467*, 2025.
- [16] Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Agentic test-time scaling for WebAgents. *arXiv preprint arXiv:2602.12276*, 2026.
- [17] Yin Lin, Elaine Ang, Erkang Zhu, Bolin Ding, and Jingren Zhou. Context as an environment: Programmatic context management for long-horizon agents. *arXiv preprint arXiv:2608.21690*, 2026.
- [18] Siyuan Ma, Bo Gao, Zikai Xiao, Hailong Wang, Xinlei Yu, Rui Qian, Jiayu Qian, Luqi Gong, and Yang Liu. CoT2-Meta: Budgeted metacognitive control for test-time reasoning. *arXiv preprint arXiv:2603.28135*, 2026.
- [19] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *International Conference on Learning Representations*, 2025.
- [20] Davide Paglieri, Bartłomiej Cupiał, Jonathan Cook, Ulyana Piterbarg, Jens Tuyls, Edward Grefenstette, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. Learning when to plan: Efficiently allocating test-time compute for LLM agents. *arXiv preprint arXiv:2509.03581*, 2025.
- [21] Dinesh Parthasarathy, Georgios Kontes, Axel Plinge, and Christopher Mutschler. C-MCTS: Safe planning with monte carlo tree search. *Workshop on Safe and Trustworthy Agents at NeurIPS*, 2024.
- [22] Peijia Qin, Qi Cao, and Pengtao Xie. ATLAS: Agentic test-time learning-to-allocate scaling. *arXiv preprint arXiv:2606.01667*, 2026.
- [23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.
- [24] Stuart J. Russell and Eric Wefald. *Do the Right Thing: Studies in Limited Rationality*. MIT Press, 1991.
- [25] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. doi: 10.1038/s41586-020-03051-4.
- [26] Eren Sezener and Peter Dayan. Static and dynamic values of computation in MCTS. In *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence*, volume 124 of *Proceedings of Machine Learning Research*, pages 31–40. PMLR, 2020. URL <https://proceedings.mlr.press/v124/sezener20a.html>.
- [27] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneshelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016. doi: 10.1038/nature16961.
- [28] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017. doi: 10.1038/nature24270.

- [29] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [30] Asterios Tsourvas, Wei Sun, and Georgia Perakis. Causal LLM routing: End-to-end regret minimization from observational data. *arXiv preprint arXiv:2505.16037*, 2025.
- [31] Ngo Anh Vien, Wolfgang Ertel, Viet-Hung Dang, and TaeChoong Chung. Monte-carlo tree search for bayesian reinforcement learning. *Applied Intelligence*, 39:345–353, 2013. doi: 10.1007/s10489-012-0416-2.
- [32] Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026. Accepted to ICML 2026.
- [33] Hang Xu, Zang Yu, Yehui Tang, Pengbo Hu, Yuhao Tang, and Hao Dong. MCTS-EP: Empowering embodied planning with online preference optimization. *arXiv preprint arXiv:2509.17116*, 2025.
- [34] Jiaqi Xue, Qian Lou, Jiarong Xing, and Heng Huang. R2-Router: A new paradigm for LLM routing with reasoning. *arXiv preprint arXiv:2602.02823*, 2026. Accepted to ICML 2026.
- [35] Jaesik Yoon, Hyeonseo Cho, Doojin Baek, Yoshua Bengio, and Sungjin Ahn. Monte carlo tree diffusion for system 2 planning. *International Conference on Machine Learning*, 2025.
- [36] King Zhu, Hanhao Li, Siwei Wu, Tianshun Xing, Dehua Ma, Xiangru Tang, Minghao Liu, Jian Yang, et al. Scaling test-time compute for LLM agents. *arXiv preprint arXiv:2506.12928*, 2025.
- [37] Yuxin Zuo, Zikai Xiao, Li Sheng, Fei Huang, Jianhong Tu, Yuxuan Liu, Tianyi Tang, Xiaomeng Hu, Yang Su, Qingfeng Lan, et al. Qwen-AgentWorld: Language world models for general agents. *arXiv preprint arXiv:2606.24597*, 2026.